```python
"""
にゃんこ大戦争風ゲーム

右側の自陣から味方ユニット（ネコ）を出撃させ，左側の敵陣を攻略するゲーム．
お金は時間とともに増え，お金とクールダウンの許す範囲でユニットを出撃できる．
ユニットは敵に当たると立ち止まって攻撃し，敵陣のHPを0にすれば勝利．

操作：
- 数字キー 1〜3 または画面下ボタン：ユニット出撃
- Wキー / 0キー または画面右下ボタン：財布レベルアップ（お金の増加速度と上限アップ）
"""

import os
import random
import sys
import time
from typing import List, Optional

import pygame as pg

# 定数定義
WIDTH, HEIGHT = 1100, 650
GROUND_Y = 500  # 地面の高さ（ユニットの足元のy座標）
FPS = 50
ENEMY_INTERVAL = 150  # 敵が出現する間隔（フレーム数）
BASE_HP = 1000

os.chdir(os.path.dirname(os.path.abspath(__file__)))

# 味方ユニットの性能（追加機能として種類を増やしやすいように辞書で管理）
CAT_SPECS = [
    {
        "name": "ネコ",
        "cost": 50,
        "hp": 100,
        "atk": 10,
        "speed": 2.0,
        "range": 20,
        "interval": 40,
        "cooldown": 100,
        "color": (255, 220, 160),
        "size": (40, 50),
    },
    {
```

```
            "name": "タンク",
            "cost": 100,
            "hp": 500,
            "atk": 6,
            "speed": 1.2,
            "range": 20,
            "interval": 50,
            "cooldown": 200,
            "color": (200, 200, 255),
            "size": (50, 70),
        },
        {
            "name": "キック",
            "cost": 150,
            "hp": 120,
            "atk": 35,
            "speed": 1.8,
            "range": 60,
            "interval": 70,
            "cooldown": 250,
            "color": (255, 150, 150),
            "size": (40, 60),
        },
    ]

    # 敵ユニットの性能
    ENEMY_SPECS = [
        {
            "name": "ワンコ",
            "hp": 120,
            "atk": 8,
            "speed": 1.5,
            "range": 20,
            "interval": 45,
            "color": (240, 240, 240),
            "size": (40, 40),
        },
        {
            "name": "ブタ",
            "hp": 500,
            "atk": 20,
            "speed": 1.0,
            "range": 20,
            "interval": 60,
            "color": (255, 170, 200),
            "size": (60, 80),
        },
    ]


    def get_font(size: int) -> pg.font.Font:
        """日本語を表示できるフォントを探して返す.

        Args:
            size (int): フォントサイズ

        Returns:
            pg.font.Font: Fontオブジェクト（見つからない場合は標準フォント）
```

```
        """
        name = pg.font.match_font(
            "meiryo,yugothic,msgothic,notosanscjkjp,hiraginosans,takaopgothic"
        )
        return pg.font.Font(name, size)


def draw_hp_bar(screen: pg.Surface, rct: pg.Rect, hp: int, max_hp: int) -> None:
    """対象の頭上にHPバーを描画する.

    Args:
        screen (pg.Surface): 画面Surface
        rct (pg.Rect): 対象のRect
        hp (int): 現在のHP
        max_hp (int): 最大HP
    """
    ratio = max(0, hp) / max_hp
    bar = pg.Rect(rct.left, rct.top - 10, rct.width, 6)
    pg.draw.rect(screen, (80, 80, 80), bar)
    pg.draw.rect(
        screen,
        (0, 200, 0),
        (bar.left, bar.top, int(bar.width * ratio), bar.height),
    )


class Base:
    """自陣・敵陣（城）に関するクラス."""

    def __init__(self, is_cat: bool) -> None:
        """引数 is_cat：Trueなら自陣（右側），Falseなら敵陣（左側）."""
        self.hp = BASE_HP
        self.max_hp = BASE_HP
        self.image = pg.Surface((80, 200))
        self.image.fill((100, 150, 255) if is_cat else (255, 120, 120))
        pg.draw.rect(self.image, (0, 0, 0), self.image.get_rect(), 3)
        self.rect = self.image.get_rect()
        self.rect.bottom = GROUND_Y
        self.rect.centerx = WIDTH - 80 if is_cat else 80

    def draw(self, screen: pg.Surface) -> None:
        """城とHPバーを描画する.

        Args:
            screen (pg.Surface): 画面Surface
        """
        screen.blit(self.image, self.rect)
        draw_hp_bar(screen, self.rect, self.hp, self.max_hp)


class Unit(pg.sprite.Sprite):
    """味方・敵のユニットに関するクラス."""

    def __init__(self, spec: dict, is_cat: bool, x: int) -> None:
        """ユニット初期化.

        Args:
            spec (dict): ユニットの性能を表す辞書
```

```
161             is_cat (bool): Trueなら味方（左へ進む），Falseなら敵（右へ進む）
162             x (int): 出現位置のx座標
163         """
164         super().__init__()
165         self.spec = spec
166         self.hp = spec["hp"]
167         self.dir = -1 if is_cat else +1
168         self.atk_timer = spec["interval"]  # 最初の1発はすぐ撃てるようにする
169         self.image = pg.Surface(spec["size"])
170         self.image.fill(spec["color"])
171         pg.draw.rect(self.image, (0, 0, 0), self.image.get_rect(), 2)
172         self.rect = self.image.get_rect()
173         self.rect.bottom = GROUND_Y
174         if is_cat:
175             self.rect.right = x
176         else:
177             self.rect.left = x
178         self.x = float(self.rect.centerx)  # 小数の速度を扱うため，x座標は別に保持する
179 
180     def can_reach(self, rct: pg.Rect) -> bool:
181         """対象のRectが攻撃射程内にあるかを判定する．
182 
183         Args:
184             rct (pg.Rect): 敵ユニットまたは敵陣のRect
185 
186         Returns:
187             bool: 射程内ならTrue
188         """
189         if self.dir < 0:
190             return rct.right >= self.rect.left - self.spec["range"]
191         return rct.left <= self.rect.right + self.spec["range"]
192 
193     def find_target(
194         self, opponents: pg.sprite.Group, base: Base
195     ) -> Optional["Unit | Base"]:
196         """攻撃対象を探す（射程内のユニットのうち最も近いもの，いなければ敵城）．
197 
198         Args:
199             opponents (pg.sprite.Group): 相手側ユニットのグループ
200             base (Base): 相手側の城
201 
202         Returns:
203             Optional[Unit | Base]: 攻撃対象（いなければNone）
204         """
205         cands = [o for o in opponents if self.can_reach(o.rect)]
206         if cands:
207             return min(cands, key=lambda o: abs(o.rect.centerx - self.rect.centerx))
208         if self.can_reach(base.rect):
209             return base
210         return None
211 
212     def update(self, opponents: pg.sprite.Group, base: Base) -> None:
213         """HP判定，攻撃，移動を行う．
214 
215         Args:
216             opponents (pg.sprite.Group): 相手側ユニットのグループ
217             base (Base): 相手側の城
218         """
```

```
            if self.hp <= 0:
                self.kill()
                return
            target = self.find_target(opponents, base)
            if target is None:
                self.x += self.dir * self.spec["speed"]
                self.rect.centerx = int(self.x)
                return
            self.atk_timer += 1
            if self.atk_timer >= self.spec["interval"]:
                target.hp -= self.spec["atk"]
                self.atk_timer = 0


class Wallet:
    """お金（所持金）および財布のレベルアップに関するクラス."""

    def __init__(self) -> None:
        """初期パラメータの設定."""
        self.money = 100.0
        self.level = 1
        self.max_level = 8
        self.max_money = 500  # 初期上限
        self.income = 0.4  # 1フレームあたりの初期増加量

    def update(self) -> None:
        """お金を時間経過で増やす（上限あり）."""
        self.money = min(float(self.max_money), self.money + self.income)

    def pay(self, cost: int) -> bool:
        """所持金が足りれば支払う.

        Args:
            cost (int): 支払う金額

        Returns:
            bool: 支払えたらTrue，足りなければFalse
        """
        if self.money < cost:
            return False
        self.money -= cost
        return True

    def get_level_up_cost(self) -> int:
        """現在のレベルに応じたレベルアップに必要な費用を取得する.

        Returns:
            int: レベルアップに必要な費用（最大レベルの場合は0）
        """
        if self.level >= self.max_level:
            return 0
        return 100 * self.level

    def level_up(self) -> bool:
        """所持金が足りれば財布レベルを上げ，所持金上限と増加速度を強化する.

        Returns:
            bool: レベルアップに成功したらTrue，失敗・レベルMAXならFalse
        """
```

```
            if self.level >= self.max_level:
                return False
            cost = self.get_level_up_cost()
            if not self.pay(cost):
                return False

            self.level += 1
            self.max_money += 350  # 所持金上限の増加
            self.income += 0.35   # 毎フレームの収入増加量アップ
            return True


    def make_buttons() -> List[pg.Rect]:
        """出撃ボタンのRectのリストを作る．

        Returns:
            List[pg.Rect]: 味方ユニットの種類ごとのボタンRect
        """
        return [pg.Rect(20 + i * 170, 540, 160, 90) for i in range(len(CAT_SPECS))]


    def make_wallet_button() -> pg.Rect:
        """財布レベルアップボタンのRectを作る．

        Returns:
            pg.Rect: 財布レベルアップボタンのRect
        """
        return pg.Rect(WIDTH - 180, 540, 160, 90)


    def try_spawn(
        idx: int,
        wallet: Wallet,
        cooldowns: List[int],
        cats: pg.sprite.Group,
        cat_base: Base,
    ) -> None:
        """お金とクールダウンの条件を満たせば，味方ユニットを出撃させる．

        Args:
            idx (int): CAT_SPECSの番号
            wallet (Wallet): 所持金オブジェクト
            cooldowns (List[int]): ユニットごとの残りクールダウン
            cats (pg.sprite.Group): 味方ユニットのグループ
            cat_base (Base): 自陣
        """
        spec = CAT_SPECS[idx]
        if cooldowns[idx] > 0 or not wallet.pay(spec["cost"]):
            return
        cats.add(Unit(spec, True, cat_base.rect.left))
        cooldowns[idx] = spec["cooldown"]


    def draw_ui(
        screen: pg.Surface,
        font: pg.font.Font,
        wallet: Wallet,
        cooldowns: List[int],
```

```python
        buttons: List[pg.Rect],
        wallet_btn: pg.Rect,
    ) -> None:
        """所持金表示・出撃ボタン・財布レベルアップボタンを描画する.

        Args:
            screen (pg.Surface): 画面Surface
            font (pg.font.Font): 表示用フォント
            wallet (Wallet): 所持金オブジェクト
            cooldowns (List[int]): ユニットごとの残りクールダウン
            buttons (List[pg.Rect]): 出撃ボタンのRectリスト
            wallet_btn (pg.Rect): 財布レベルアップボタンのRect
        """
        # ユニット出撃ボタン描画
        for i, (spec, btn) in enumerate(zip(CAT_SPECS, buttons)):
            usable = cooldowns[i] == 0 and wallet.money >= spec["cost"]
            pg.draw.rect(screen, (255, 230, 120) if usable else (150, 150, 150), btn)
            if cooldowns[i] > 0:  # クールダウン中は残り時間を暗いゲージで示す
                h = int(btn.height * cooldowns[i] / spec["cooldown"])
                pg.draw.rect(screen, (90, 90, 90), (btn.left, btn.top, btn.width, h))
            pg.draw.rect(screen, (0, 0, 0), btn, 3)
            name = font.render(f"{i + 1}:{spec['name']}", True, (0, 0, 0))
            cost = font.render(f"{spec['cost']}円", True, (0, 0, 0))
            screen.blit(name, (btn.left + 10, btn.top + 10))
            screen.blit(cost, (btn.left + 10, btn.top + 50))

        # 財布レベルアップボタン描画
        cost_level_up = wallet.get_level_up_cost()
        is_max = wallet.level >= wallet.max_level
        can_level_up = not is_max and wallet.money >= cost_level_up

        pg.draw.rect(
            screen, (100, 220, 100) if can_level_up else (160, 160, 160), wallet_btn
        )
        pg.draw.rect(screen, (0, 0, 0), wallet_btn, 3)

        btn_lbl = font.render("[W/0]財布", True, (0, 0, 0))
        if is_max:
            cost_lbl = font.render("レベル MAX", True, (0, 0, 0))
        else:
            cost_lbl = font.render(f"{cost_level_up}円", True, (0, 0, 0))

        screen.blit(btn_lbl, (wallet_btn.left + 8, wallet_btn.top + 10))
        screen.blit(cost_lbl, (wallet_btn.left + 10, wallet_btn.top + 50))

        # 所持金および財布LV表示
        money_str = f"お金 {int(wallet.money)} / {wallet.max_money}円 (Lv.{wallet.level})"
        money = font.render(money_str, True, (0, 0, 0))
        screen.blit(money, (WIDTH - money.get_width() - 20, 20))


    def draw_field(screen: pg.Surface) -> None:
        """背景（空と地面）を描画する.

        Args:
            screen (pg.Surface): 画面Surface
        """
        screen.fill((135, 206, 235))
```

```
        pg.draw.rect(screen, (150, 110, 70), (0, GROUND_Y, WIDTH, HEIGHT - GROUND_Y))
        pg.draw.line(screen, (0, 0, 0), (0, GROUND_Y), (WIDTH, GROUND_Y), 3)



def show_result(screen: pg.Surface, font: pg.font.Font, message: str) -> None:
    """勝敗の結果を3秒間表示する.

    Args:
        screen (pg.Surface): 画面Surface
        font (pg.font.Font): 表示用フォント
        message (str): 表示する文字列
    """
    overlay = pg.Surface((WIDTH, HEIGHT))
    overlay.set_alpha(150)
    screen.blit(overlay, (0, 0))
    text = font.render(message, True, (255, 255, 255))
    screen.blit(text, text.get_rect(center=(WIDTH // 2, HEIGHT // 2)))
    pg.display.update()
    time.sleep(3)



def main() -> None:
    """メインループ関数."""
    pg.display.set_caption("ネコ大戦争（仮）")
    screen = pg.display.set_mode((WIDTH, HEIGHT))
    font = get_font(24)
    big_font = get_font(90)
    clock = pg.time.Clock()

    cat_base, enemy_base = Base(True), Base(False)
    cats, enemies = pg.sprite.Group(), pg.sprite.Group()
    wallet = Wallet()
    cooldowns = [0] * len(CAT_SPECS)
    buttons = make_buttons()
    wallet_btn = make_wallet_button()
    tmr = 0

    while True:
        for event in pg.event.get():
            if event.type == pg.QUIT:
                return

            if event.type == pg.KEYDOWN:
                # 数字キー 1〜3 で出撃
                if pg.K_1 <= event.key < pg.K_1 + len(CAT_SPECS):
                    try_spawn(event.key - pg.K_1, wallet, cooldowns, cats, cat_base)
                # Wキーまたは 0 キーで財布レベルアップ
                elif event.key in (pg.K_w, pg.K_0, pg.K_KP0):
                    wallet.level_up()

            if event.type == pg.MOUSEBUTTONDOWN and event.button == 1:
                # 出撃ボタン判定
                for i, btn in enumerate(buttons):
                    if btn.collidepoint(event.pos):
                        try_spawn(i, wallet, cooldowns, cats, cat_base)
                # 財布レベルアップボタン判定
                if wallet_btn.collidepoint(event.pos):
                    wallet.level_up()
```

```
            wallet.update()
            for i in range(len(cooldowns)):
                cooldowns[i] = max(0, cooldowns[i] - 1)
            if tmr % ENEMY_INTERVAL == ENEMY_INTERVAL - 1:
                spec = random.choices(ENEMY_SPECS, weights=[3, 1])[0]
                enemies.add(Unit(spec, False, enemy_base.rect.right))
            cats.update(enemies, enemy_base)
            enemies.update(cats, cat_base)

            draw_field(screen)
            enemy_base.draw(screen)
            cat_base.draw(screen)
            cats.draw(screen)
            enemies.draw(screen)
            for unit in list(cats) + list(enemies):
                if isinstance(unit, Unit):
                    draw_hp_bar(screen, unit.rect, unit.hp, unit.spec["hp"])
            draw_ui(screen, font, wallet, cooldowns, buttons, wallet_btn)

            if enemy_base.hp <= 0:
                show_result(screen, big_font, "勝利！")
                return
            if cat_base.hp <= 0:
                show_result(screen, big_font, "敗北…")
                return
            pg.display.update()
            tmr += 1
            clock.tick(FPS)


    if __name__ == "__main__":
        pg.init()
        main()
        pg.quit()
        sys.exit()
```